CLIENT SECURITY BRIEFING

Security, Privacy & Data Protection

How KingFisher Wings protects passwords, isolates every company's data, and manages confidentiality, integrity, and availability across the platform.

Product
KingFisher Wings ERP (Freight SaaS)

Audience
Prospects · Customers · Compliance reviews

Document type
Technical overview with illustrative examples

Date
November 2026

Confidential — for authorized client review. Example credentials below are fictional and used only for illustration.

1. Security principles we build on

Our platform follows industry practice: never store usable passwords, never trust the client alone, and isolate every tenant's data at the database layer.

Confidentiality

Only the right people see the right data — per company, tenant, role, and portal.

Integrity

Validated inputs, audited changes, and controlled writes protect accuracy of freight records.

Availability

Rate limits, session controls, and resilient hosting keep operations running safely.

Passwords

Argon2id hashing — irreversible. Plain text is never saved.

Tenancy

PostgreSQL Row-Level Security + application tenant context.

Access

Separate JWT principals for staff, customer portal, vendor portal, and Super Admin.

Defense in depth (layers)

Edge	HTTPS · Helmet security headers · API rate limiting (Throttler)
Identity	Argon2id passwords · JWT access tokens · server-side sessions (`jti`) for instant revoke
Authorization	Role + permission catalog · portal/vendor document rights · Super Admin platform scope only
Data plane	`runWithTenant` + PostgreSQL RLS — a missed filter cannot leak another company
Accountability	Audit logs · staff activity alerts · Super Admin tech logs for issue triage

2. Password security — from typing to database

When a user sets or changes a password, the application **never stores the password itself**. It runs a one-way cryptographic hash (Argon2id) and stores only the result.

> **Illustrative example** (fictional user — not a live account)
> User types: `Welcome@123` → system saves only the Argon2id hash shown below.

1 User enters password — Login / invite / reset form → **2 PasswordUtil.hash()** — Argon2id · memory-hard → **3 Store hash only** — `password_hash` column → **4 On login: verify** — Compare input vs hash

Plain text (never saved)

```
Welcome@123
```

What is saved in the database

```
$argon2id$v=19$m=65536,t=3,p=1$fscbKfXqzPsE+/5mtcKqDw$kYIllLFe2ETG10f7GpHPT11jkD5V0r6o2MptmoYjRSQ
```

Parameter	Our setting	Why it matters
Algorithm	**Argon2id**	Modern password hashing (winner of the Password Hashing Competition). Resists GPU and side-channel attacks better than older hashes.
memoryCost	65,536 KiB	Makes brute-force expensive in memory — slows attackers even with powerful hardware.
timeCost	3 iterations	Balances login speed for users with resistance for attackers.
parallelism	1	Stable, predictable hashing cost per request.

What an attacker cannot do

- Read your real password from the database
- "Decrypt" the hash back to `Welcome@123`
- Reuse a stolen hash as a login password

What happens at login

- User submits password again
- `PasswordUtil.verify(hash, password)`
- Match → issue JWT + create session
- No match → reject (lockout / throttle apply)

3. Object-oriented design around passwords

Password handling is encapsulated in a single utility class so hashing rules stay consistent everywhere (staff, tenant owner, Super Admin, portal invite temp passwords, refresh-token hashes).



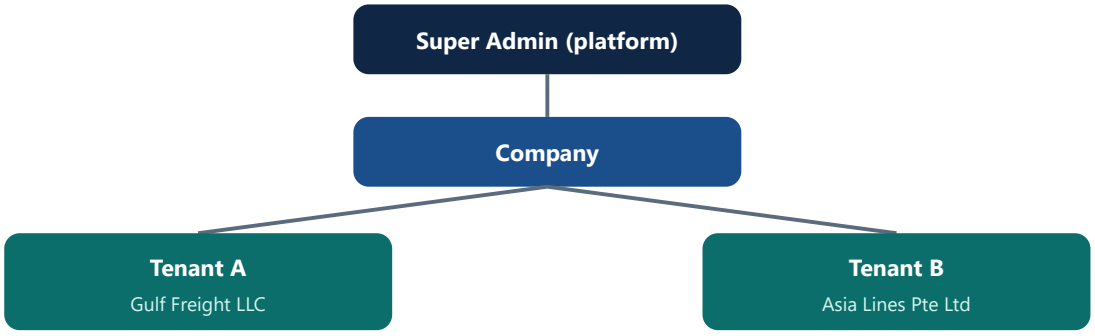
OOP concept	How we use it for security
Encapsulation	Hashing parameters live inside `PasswordUtil`. Callers never invent their own algorithm.
Abstraction	Services call `hash` / `verify` — they do not handle salts or Argon2 encoding.
Single responsibility	Auth services handle sessions & JWT; password crypto stays in one class.
Reuse without copy-paste	Same class protects staff passwords, tenant-owner passwords, Super Admin, and hashed refresh tokens.

> **Important client message:** Even platform administrators and DB operators cannot recover a user's original password from the stored hash.

4. Keeping every company’s data separate

KingFisher Wings is multi-tenant SaaS. Hierarchy is enforced in product and in the database:

Super Admin → Company → Tenant → Users / Portals



✕ RLS WALL — Tenant A queries cannot see Tenant B rows ✕

Tenant A data
- Jobs · Quotes · Invoices
- Parties · Users · Docs
- `tenant_id = A`

Tenant B data
- Jobs · Quotes · Invoices
- Parties · Users · Docs
- `tenant_id = B`

Platform only
- Companies · Tenants
- Super Admin console
- No tenant job leakage

How isolation works (two locks)

Layer	Mechanism	Client benefit
Application	`prisma.runWithTenant(tenantId, …)` sets tenant context for every request	Business logic always scoped to the logged-in company
Database (RLS)	PostgreSQL Row-Level Security: `tenant_id = current_tenant_id()`	Even a coding mistake cannot return another tenant’s rows

> **Example:** User from *Gulf Freight LLC* opens Jobs. The API sets tenant context to A. Postgres RLS allows only rows where `tenant_id = A`. Asia Lines’ shipments never appear — not in UI, not in API JSON, not in exports.

5. Who can access what — identity isolation

Different actors receive different token types. A customer portal token cannot call staff ERP APIs, and a staff token cannot act as a vendor.

Actor	Login	Principal	Sees
Super Admin	`/auth/super-admin/login`	`super_admin`	Platform: companies & tenants (not tenant ops data via staff APIs)
Tenant Admin / Staff	`/auth/login` or tenant-login	`user`	Own tenant ERP modules per role & permissions
Customer (portal)	`/portal/auth/*`	`portal`	Own party's shipments, AR, quotes — stripped of internal cost/GP
Vendor (portal)	`/vendor/auth/*`	`vendor`	Own vendor AP documents — separate from customer portal



Session safety

- Access JWT embeds permissions for speed
- Server session row validated every request
- Logout / password change revokes immediately via `jti`
- Refresh tokens stored as hashes (same Argon2 discipline)

Extra controls

- Login throttling on auth endpoints
- Optional IP / office-hours / lockout policies
- Password history on change
- TOTP 2FA implemented — enable at production go-live

6. Privacy, safety & operational controls

Privacy by design

- Portal/vendor responses strip internal notes, GL accounts, other parties, cost & GP
- Document rights matrices (`PortalPermission` / `VendorPermission`)
- Public track links use opaque tokens — barcode labels do not expose ERP data
- Soft-delete keeps auditability without silent hard wipes

Safety & abuse resistance

- Input validation (DTO whitelist / transform)
- Helmet HTTP headers reduce common web risks
- Throttling limits brute-force and API flood
- Permission checks on mutating business actions

Illustrative request path

1. **HTTPS request** — TLS in transit
2. **Throttle + JWT** — Who is calling?
3. **Permissions** — May they do this?
4. **Tenant + RLS** — Only their rows
5. **Audit** — Who changed what

Concern	How KingFisher Wings addresses it
Password theft from DB dump	Argon2id hashes only — no reversible encryption of passwords
Cross-company data leak	Tenant context + PostgreSQL RLS on business tables
Stolen access token	Short-lived JWT + server session revoke on logout/password change
Customer seeing vendor/cost data	Separate principals + response stripping + document rights
Insider / ops mistakes	Role permissions, audit logs, Super Admin tech issue tracking

7. Security certificates — what Fresa shows & how we can obtain them

Fresa Technologies publicly lists management-system certifications. These are **organization / process certificates** (how the company runs security & quality), not automatic "badges" generated by source code alone.

ISO/IEC 27001:2022

Information Security (ISMS)

Policies, risk assessment, access control, incident response, supplier security, continual audit. This is the primary certificate clients ask for in SaaS / ERP deals.

ISO 9001:2015

Quality Management (QMS)

Documented delivery processes, customer satisfaction, corrective actions, consistent operations.

ISO/IEC 42001:2023

AI Management (AIMS)

Governance for responsible AI — relevant when AI features are productized (Fresa highlights this).

How to get certificates like Fresa (practical roadmap)

1. **Choose scope** — e.g. "Design, development, hosting operations, and support of KingFisher Wings freight ERP."
2. **Build the management system** — information security policy, asset inventory, risk register, access-control policy, change management, backup/DR, incident response, vendor list, staff training records.
3. **Implement controls** — map our existing technical controls (Argon2id, RLS, JWT isolation, logging, backups) into ISO 27001 Annex A / Statement of Applicability.
4. **Operate for evidence** — typically several months of real tickets, access reviews, backup tests, vulnerability handling, and management reviews.
5. **Internal audit + management review** — fix gaps before the external auditor arrives.
6. **Select an accredited certification body** — IAF-recognized body (examples in market: InterCert, BSI, TÜV, SGS, Bureau Veritas, local IAS-accredited CBs). Fresa's public materials reference InterCert / IQNet-related pathways.
7. **Stage 1 + Stage 2 audit** — documentation review, then on-site/remote effectiveness audit → certificate (usually 3-year cycle with annual surveillance).
8. **Optional parallel tracks** — ISO 9001 for quality; ISO 42001 if AI features need formal AI governance; SOC 2 Type I/II if US enterprise buyers require it (AICPA Trust Services Criteria via a CPA firm).

Track	Typical buyer ask	Rough effort*
ISO/IEC 27001	Security & privacy RFPs, UAE/EU enterprise	3–9 months + surveillance years
ISO 9001	Delivery quality / process maturity	2–6 months
ISO 42001	AI feature governance marketing	After AI product processes exist
SOC 2 Type II	US SaaS procurement	Often 6–12+ months observation

*Depends on company size, existing policies, and whether you hire a consultant. Costs vary widely by certification body and geography.

Honest positioning for clients today: KingFisher Wings already implements strong *technical* security controls described in this document. Formal ISO/SOC certificates are an organizational next step — we can pursue ISO/IEC 27001 first to match the market expectation set by peers such as Fresa.

8. Summary for decision makers

You can tell your stakeholders

- Passwords are hashed with Argon2id — never stored in plain text
- Each company's freight data is isolated with application tenancy *and* database RLS
- Staff, customers, vendors, and platform admins use separate identity channels
- Security is layered: network headers, auth, permissions, data isolation, audit

Recommended next steps with us

- Enable production 2FA for admins at go-live
- Agree backup / DR and hosting region commitments
- Start ISO/IEC 27001 gap assessment if certificate is required in contract
- Optional: SOC 2 if selling heavily into US enterprise

Password journey — one-page reminder

```
User types:  Welcome@123
        │
        ▼
PasswordUtil.hash()  →  Argon2id (m=65536, t=3, p=1)
        │
        ▼
Database stores ONLY:
$argon2id$v=19$m=65536,t=3,p=1$…$…
        │
        ▼
Login later: PasswordUtil.verify(storedHash, typedPassword)
        │
        ├── match → JWT + session
        └── fail  → reject (throttle / lockout)
```

KingFisher Wings — Security, Privacy & Data Protection Overview · November 2026

Technical controls described reflect the current NestJS + PostgreSQL (RLS) + Argon2id architecture. Certification claims require a completed audit by an accredited body and are not asserted by this document. Example password and hash are illustrative. © KingFisher Wings / project owners.